

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division:- C Batch :- 2 Practical No :- 6

Roll no :- 46 PRN No :- 12412632 Name of Student :- Krishna Meghwani

Subject Name : Deep Learning

Problem Statement :- Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Theory

1. What is CNN?

A Convolutional Neural Network (CNN) is a deep learning algorithm mainly used for image classification and image processing.

In this experiment, the CNN takes a tomato leaf image as input and predicts its disease class.

2. Convolution Layer

The convolution layer extracts important features from the input image using small filters called kernels.

3. ReLU Activation

ReLU stands for Rectified Linear Unit.

Formula:

$$\text{ReLU}(x) = \max(0, x)$$

It converts negative values to zero and keeps positive values.

It helps the CNN learn complex patterns.

4. Max Pooling

Max pooling reduces the size of the feature map while retaining important information.

We use:

MaxPooling2D(2×2)

5. Softmax

The final output layer uses the Softmax activation function.

Since Tomato dataset contains 10 classes, we use:

Dense(10, activation='softmax')

Softmax gives a probability for each disease class.

For example:

Bacterial Spot	0.05
Early Blight	0.82
Healthy	0.03
Late Blight	0.06
...	

The class with the highest probability becomes the predicted class.

Algorithm

Algorithm: CNN-Based Tomato Disease Classification

Step 1: Load the Tomato Disease Dataset containing training and validation images.

Step 2: Identify the different disease classes present in the dataset.

Step 3: Resize all input images to a fixed size, such as **128 × 128 pixels**.

Step 4: Normalize the pixel values from **0–255 to 0–1**.

Step 5: Load the training and validation datasets using TensorFlow/Keras.

Step 6: Create a CNN model.

Step 7: Add a convolution layer with **32 filters** and apply ReLU activation.

Step 8: Add a **2 × 2 Max Pooling** layer.

Step 9: Add a convolution layer with **128 filters** and apply ReLU activation.

Step 10: Add a Dense layer containing **128 neurons** with ReLU activation.

Step 11: Add a Dropout layer to reduce overfitting.

Step 12: Train the CNN using the training dataset for a specified number of epochs.

Step 13: Validate the model using the validation dataset.

Step 14: Calculate the validation accuracy and loss.

Step 15: Plot the training and validation accuracy and loss graphs.

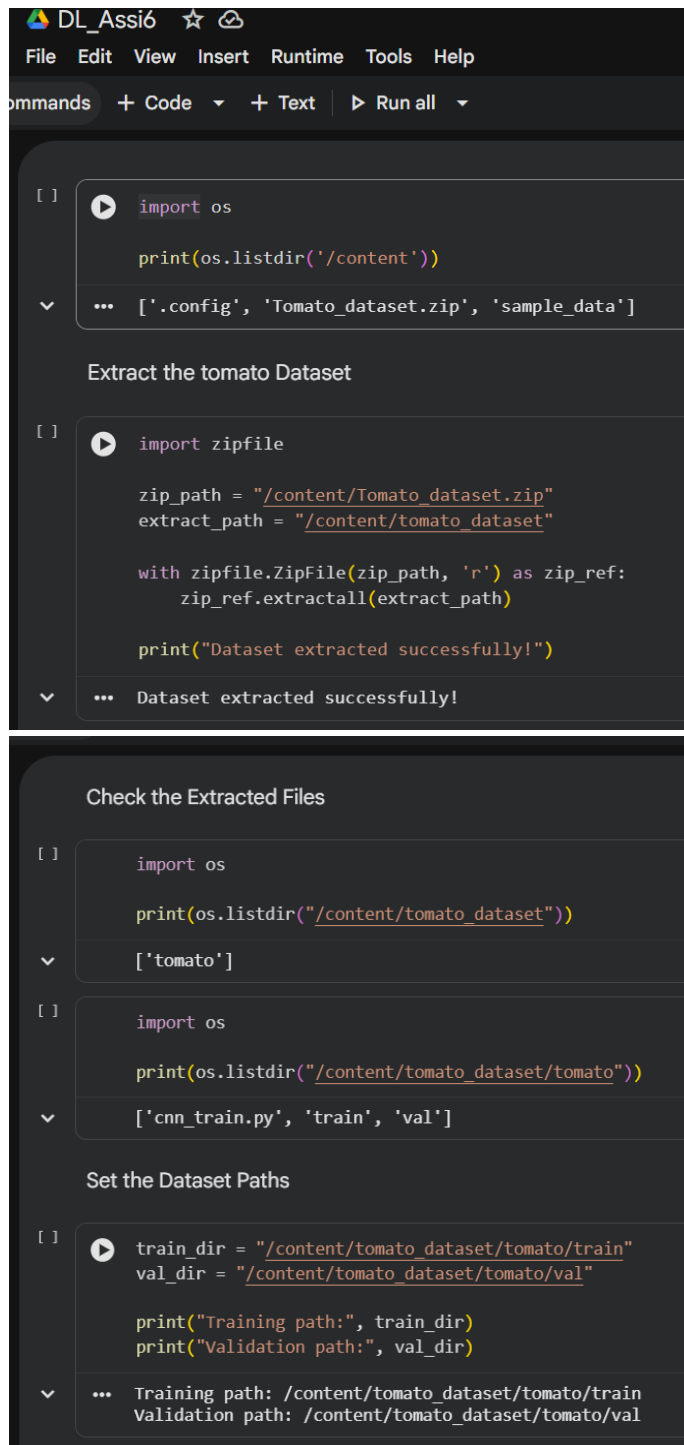
Step 16: Generate a classification report containing precision, recall, and F1-score.

Evaluation Measures

The CNN performance is evaluated using:

- 1. Accuracy** – Percentage of correctly classified images.
 - 2. Loss** – Measures the error between actual and predicted classes.
 - 3. Confusion Matrix** – Shows correct and incorrect predictions for each disease.
 - 4. Precision** – Measures how many predicted positive samples are actually correct.
 - 5. Recall** – Measures how many actual positive samples are correctly identified.
 - 6. F1-score** – Harmonic mean of precision and recall.
-

Code :-



The screenshot shows a Jupyter Notebook with the following sections and code:

Commands + Code + Text ▶ Run all

[] ▶ `import os`

`print(os.listdir('/content'))`

... `['.config', 'Tomato_dataset.zip', 'sample_data']`

Extract the tomato Dataset

[] ▶ `import zipfile`

`zip_path = "/content/Tomato_dataset.zip"`
`extract_path = "/content/tomato_dataset"`

`with zipfile.ZipFile(zip_path, 'r') as zip_ref:`
`zip_ref.extractall(extract_path)`

`print("Dataset extracted successfully!")`

... `Dataset extracted successfully!`

Check the Extracted Files

[] `import os`

`print(os.listdir("/content/tomato_dataset"))`

... `['tomato']`

[] `import os`

`print(os.listdir("/content/tomato_dataset/tomato"))`

... `['cnn_train.py', 'train', 'val']`

Set the Dataset Paths

[] ▶ `train_dir = "/content/tomato_dataset/tomato/train"`
`val_dir = "/content/tomato_dataset/tomato/val"`

`print("Training path:", train_dir)`
`print("Validation path:", val_dir)`

... `Training path: /content/tomato_dataset/tomato/train`
`Validation path: /content/tomato_dataset/tomato/val`

Check the Tomato Disease Classes

```
import os

class_folders = os.listdir(train_dir)

print("Number of classes:", len(class_folders))
print("\nClasses:")

for i, folder in enumerate(class_folders):
    print(i + 1, ":", folder)

... Number of classes: 10

Classes:
1 : Tomato__Bacterial_spot
2 : Tomato__Early_blight
3 : Tomato__healthy
4 : Tomato__Late_blight
5 : Tomato__Spider_mites Two-spotted_spider_mite
6 : Tomato__Tomato_mosaic_virus
7 : Tomato__Septoria_leaf_spot
8 : Tomato__Tomato_Yellow_Leaf_Curl_Virus
9 : Tomato__Target_Spot
10 : Tomato__Leaf_Mold
```

Count Images in Each Class

```
import os

print("Training Dataset:")

total_train = 0

for folder in class_folders:
    folder_path = os.path.join(train_dir, folder)

    image_count = len([
        file for file in os.listdir(folder_path)
        if file.lower().endswith(('.jpg', '.jpeg', '.png'))
    ])

    print(folder, ":", image_count)

    total_train += image_count

print("\nTotal Training Images:", total_train)

... Training Dataset:
Tomato__Bacterial_spot : 1000
Tomato__Early_blight : 1000
Tomato__healthy : 1000
Tomato__Late_blight : 1000
Tomato__Spider_mites Two-spotted_spider_mite : 1000
Tomato__Tomato_mosaic_virus : 1000
Tomato__Septoria_leaf_spot : 1000
Tomato__Tomato_Yellow_Leaf_Curl_Virus : 1000
Tomato__Target_Spot : 1000
Tomato__Leaf_Mold : 1000

Total Training Images: 10000
```

Count Validation Images

```
print("Validation Dataset:")

total_val = 0

for folder in class_folders:
    folder_path = os.path.join(val_dir, folder)

    image_count = len([
        file for file in os.listdir(folder_path)
        if file.lower().endswith(('.jpg', '.jpeg', '.png'))
    ])

    print(folder, ":", image_count)

    total_val += image_count

print("\nTotal Validation Images:", total_val)
```

... Validation Dataset:

Tomato__Bacterial_spot	: 100
Tomato__Early_blight	: 100
Tomato__healthy	: 100
Tomato__Late_blight	: 100
Tomato__Spider_mites Two-spotted_spider_mite	: 100
Tomato__Tomato_mosaic_virus	: 100
Tomato__Septoria_leaf_spot	: 100
Tomato__Tomato_Yellow_Leaf_Curl_Virus	: 100
Tomato__Target_Spot	: 100
Tomato__Leaf_Mold	: 100

Total Validation Images: 1000

Load the Training Dataset

```
import tensorflow as tf

train_dataset = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=(128, 128),
    batch_size=32,
    shuffle=True
)
```

Found 10000 files belonging to 10 classes.

Load the Validation Dataset

```
val_dataset = tf.keras.utils.image_dataset_from_directory(
    val_dir,
    image_size=(128, 128),
    batch_size=32,
    shuffle=False
)
```

... Found 1000 files belonging to 10 classes.

Get Class Names from TensorFlow

```
class_names = train_dataset.class_names

print("Number of Classes:", len(class_names))

print("\nClass Names:")

for i, name in enumerate(class_names):
    print(i, ":", name)

... Number of Classes: 10

Class Names:
0 : Tomato_Bacterial_spot
1 : Tomato_Early_blight
2 : Tomato_Late_blight
3 : Tomato_Leaf_Mold
4 : Tomato_Septoria_leaf_spot
5 : Tomato_Spider_mites_Two-spotted_spider_mite
6 : Tomato_Target_Spot
7 : Tomato_Tomato_Yellow_Leaf_Curl_Virus
8 : Tomato_Tomato_mosaic_virus
9 : Tomato_healthy
```

Visualize Tomato Leaf Images

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))

for images, labels in train_dataset.take(1):

    for i in range(9):

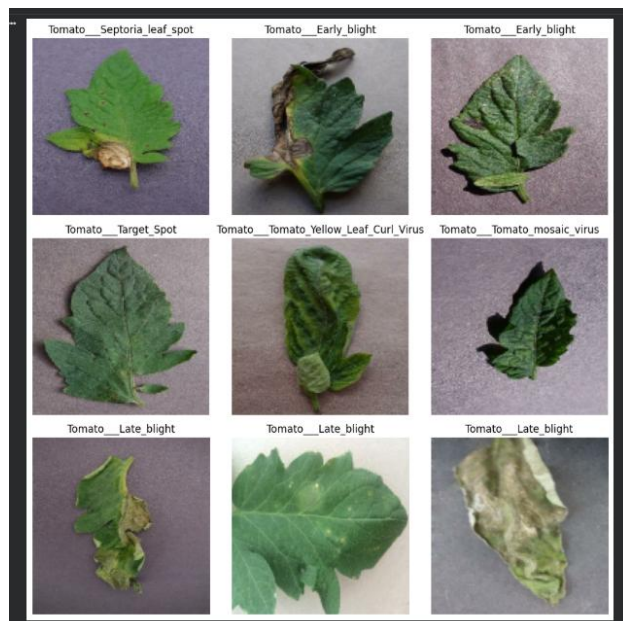
        plt.subplot(3, 3, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))

        plt.title(class_names[labels[i]])

        plt.axis("off")

plt.tight_layout()
plt.show()
```



```

import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))

for images, labels in train_dataset.take(1):

    for i in range(9):

        plt.subplot(3, 3, i + 1)

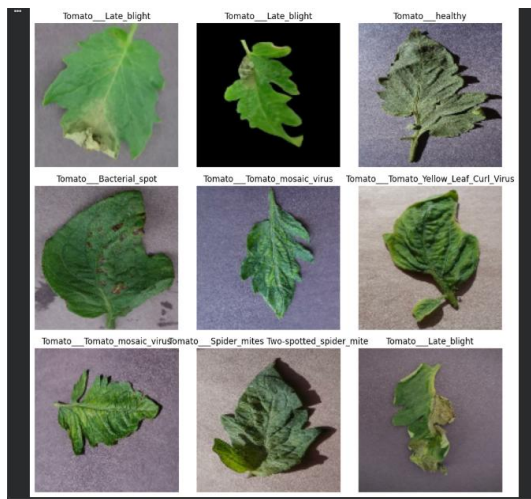
        plt.imshow(images[i].numpy().astype("uint8"))

        plt.title(class_names[labels[i]])

        plt.axis("off")

plt.tight_layout()
plt.show()

```



```

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Rescaling
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout

print("CNN libraries imported successfully!")

```

... CNN libraries imported successfully!

Create CNN Model

```
model = Sequential()

# Normalize pixel values from 0-255 to 0-1
model.add(
    Rescaling(
        1./255,
        input_shape=(128, 128, 3)
    )
)

# Convolution Layer 1
model.add(
    Conv2D(
        32,
        (3, 3),
        activation='relu'
    )
)

# Max Pooling Layer 1
model.add(
    MaxPooling2D(
        (2, 2)
    )
)

# Convolution Layer 2
model.add(
    Conv2D(
        64,
        (3, 3),
        activation='relu'
    )
)
```

```
# Max Pooling Layer 2
model.add(
    MaxPooling2D(
        (2, 2)
    )
)

# Convolution Layer 3
model.add(
    Conv2D(
        128,
        (3, 3),
        activation='relu'
    )
)

# Max Pooling Layer 3
model.add(
    MaxPooling2D(
        (2, 2)
    )
)

# Flatten Layer
model.add(
    Flatten()
)

# Fully Connected Layer
model.add(
    Dense(
        128,
        activation='relu'
    )
)

# Dropout Layer
model.add(
    Dropout(0.5)
)
```

```
# Output Layer - 10 classes
model.add(
    Dense(
        10,
        activation='softmax'
    )
)

print("CNN model created successfully!")
```

CNN model created successfully!

model.summary()

Model: "sequential_1"

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 128, 128, 3)	0
conv2d_3 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_4 (Conv2D)	(None, 61, 61, 64)	10,496
max_pooling2d_4 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_5 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_1 (Flatten)	(None, 25088)	0
dense_2 (Dense)	(None, 128)	3,211,392
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 10)	1,290

Total params: 3,305,930 (12.61 MB)
Trainable params: 3,305,930 (12.61 MB)
Non-trainable params: 0 (0.00 B)

Compile CNN Model

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("Model compiled successfully!")
```

Model compiled successfully!

Train CNN Model

```
history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=10
)
```

Epoch 1/10
313/313 ————— 344s 1s/step - accuracy: 0.4719 - loss: 1.5007 - val_accuracy: 0.6180 - val_loss: 1.0244
Epoch 2/10
313/313 ————— 391s 1s/step - accuracy: 0.7040 - loss: 0.8700 - val_accuracy: 0.7800 - val_loss: 0.6644
Epoch 3/10
313/313 ————— 374s 1s/step - accuracy: 0.7681 - loss: 0.6740 - val_accuracy: 0.7660 - val_loss: 0.7119
Epoch 4/10
313/313 ————— 376s 1s/step - accuracy: 0.8088 - loss: 0.5607 - val_accuracy: 0.8410 - val_loss: 0.5343
Epoch 5/10
313/313 ————— 381s 1s/step - accuracy: 0.8281 - loss: 0.4941 - val_accuracy: 0.8650 - val_loss: 0.4188
Epoch 6/10
313/313 ————— 396s 1s/step - accuracy: 0.8594 - loss: 0.4100 - val_accuracy: 0.8560 - val_loss: 0.4336
Epoch 7/10
313/313 ————— 390s 1s/step - accuracy: 0.8749 - loss: 0.3741 - val_accuracy: 0.8590 - val_loss: 0.4702
Epoch 8/10
313/313 ————— 384s 1s/step - accuracy: 0.8838 - loss: 0.3495 - val_accuracy: 0.8730 - val_loss: 0.4141
Epoch 9/10
313/313 ————— 369s 1s/step - accuracy: 0.8889 - loss: 0.3175 - val_accuracy: 0.8800 - val_loss: 0.4183
Epoch 10/10
313/313 ————— 348s 1s/step - accuracy: 0.9056 - loss: 0.2751 - val_accuracy: 0.8240 - val_loss: 0.5872

Conclusion :-

The Convolutional Neural Network (CNN) was successfully designed and implemented using TensorFlow/Keras for tomato leaf disease classification. The CNN automatically extracted important features from the images using convolution and pooling layers and classified the tomato leaves into different

disease classes. The model performance was evaluated using accuracy, loss, and confusion matrix, and the results showed that CNN can be effectively used for tomato disease image classification.